

An Agentic AI Framework for Conversational Sports ERP Management with Constraint-Aware Scheduling, Risk-Gated Actions, and Auditable Governance

Divyani Denge

Department of Computer Science and Engineering, Faculty of Engineering and Technology
Ramdeobaba University, Nagpur, Maharashtra, India

Email: divyanidenge@gmail.com | Repository: <https://github.com/divyanidenge/Sports-ERP-AI-Agent-Advanced>

Abstract— The ERP systems that handle university sports facilities have traditionally been implemented using rigid form-driven user interfaces that do not allow flexible conversation and give cryptic responses upon failing to satisfy a booking request due to conflicts. In this paper we describe the implementation of an agentic artificial intelligence architecture for the management of a Sports ERP Management System where the conversation and the symbolic, deterministic verification of booking constraints are seamlessly coupled. We contribute the following interrelated architectural elements: (1) Constraint Engine and Deletion-based MUS/MCS Solver providing contrastive explanations and rank-ordered list of alternative booking slots if the booking request cannot be satisfied while taking into account eight domain invariants; (2) Risk-Gating Function, $\Gamma(q, S)$, that forwards the read-only requests to Autonomous Execution, the ambiguous requests to Candidate Disambiguation and the state-changing actions to Two-Step Confirmation; (3) Append-only Hash-Chained Provenance Ledger for the detection of tampering attacks on booking related state changes; (4) Planner–Executor–Reflector–Responder (PERR) Layer of multiagent orchestration using Typed Sprint Contracts and bounded ($K \leq 1$) replanning; and (5) Role-based Access Control with Read-Only SQL Sandbox that limits the privileges of students and administrators and prevents from non-SELECT operations against the database. The implementation was validated using a comprehensive suite of 166 assertions covering unit tests, integration tests, tests against live database, API tests, and conversational scenario tests, all of which have passed at the moment of testing. The self-conducted comparison against ReAct, Text-to-SQL, and monolithic function calling approaches has been carried out within the scope of the project but cannot be independently re-verified for this manuscript and is thus provided with the above caveat. We discuss the scope and limitations of our single-node implementation, the size of the evaluation benchmark, and the directions for future work.

Index Terms— *Agentic ERP; multi-agent orchestration; constraint satisfaction; minimal unsatisfiable subsets; SHA-256 hash-chained provenance; role-based access control; conversational AI*

I. INTRODUCTION

The allocation of such resources as badminton courts, swimming pools, tennis and basketball courts, and turf grounds in university sports complexes and business facilities follows certain rules which consider the timing aspects, organization-specific rules, and roles of users. Such rules are the maximum working hours of a resource, the number of reservations for each day or advanced booking, sport-to-resource compatibility, and differences between student and admin rights [1], [2]. Currently, facility bookings in many

organizations are processed via static forms on web pages or via ERP modules that lack conversational capabilities and respond with generic rejection when a user tries to book a resource violating some of the rules [3].

Large language models (LLMs) and tool-augmented conversational agents provide a great opportunity to increase the level of naturalness of such interaction [4], [5]. However, in turn, this approach poses certain risks because of connecting a language model to a transactional enterprise database: (i) language model outputs can diverge from the true state of a database, so a confident answer is not always a grounded one; (ii) an ambiguous instruction could cause an irreversible change to the state of a database; (iii) in poorly isolated systems one user's conversational session would reveal another user's bookings; (iv) finally, the absence of some sort of an audit trail would prevent the detection of whether a change was made due to a user or an agent itself [6], [7], [8]. In this paper, we describe the design and testing of the Sports ERP AI Management System that considers these issues using deterministic rule checking, confidence-based risk gating, role-based access control, and cryptographic audit logs.

II. RELATED WORK

The related works have been selected from four general areas that have some overlap among them: agentic enterprise systems, tool-aided querying, neuro-symbolic constraint verification, and access control along with provenance.

A. Agentic Enterprise Resource Planning

In their paper, Liu et al. [1] talk about an architecture called Agentic ERP that uses multiple agents and LLMs for autonomous functioning of enterprises; our system has adopted the concept of using role-specialized agents for problem-solving, though we have applied it in the area of sports facilities scheduling with additional constraints through typed Sprint contracts. Examples of the framework used in multi-agents systems like MetaGPT [6], and AutoGen [7] indicate the efficiency of assigning different roles to agents.

B. Query Processing with Tools

Kim et al. [2], in the FinAI Data Assistant, demonstrate that structured function calling is superior to Text-to-SQL translation in terms of schema correctness and execution

safety in a financial dataset scenario; the current application takes a similar approach through using typed function schemas instead of unconstrained SQL query formulation for any data modifying command. Schick et al. [9] and Patil et al. [10] further prove in general that parameterized tool interfaces help lessen the workload on the language model to construct syntactically and semantically correct queries, which explains the use of deterministic, strongly-typed tool schemas along with restricted SQL sandbox environment presented in Section V.

C. Neuro-Symbolic Constraint Checking

TRACE-CS is a hybrid logic-LLM system developed by Vasileiou and Yeoh [3] for explainable course scheduling, and it utilizes the language interface in conjunction with symbolic invariant checking; neurosymbolic integration paradigms [17] provide the foundation for balancing formal guarantees with conversational accessibility. Similar hybrid architecture with propositional, deletion-based core reduction approaches [11] used for Minimal Unsatisfiable Subsets extraction of eight facility booking invariants is presented in this paper instead of enforcing scheduling rules via the LLM.

D. Access Control and Provenance

The role-based access control model by Ferraiolo et al. [12] defines the theoretical background for the student/administrator distinction in the current application. Schneier and Kelsey [13] and Ibrahim and Li [14] emphasize the importance of cryptographically auditable trails and governance overlays for the actions of autonomous agents. Greshake et al. [8] report on the indirect prompt injection attacks on LLM-driven applications, which explains why any database modifications proposed by the LLM are considered untrusted input until passing the checks discussed further in this paper.

III. RESEARCH GAP AND MOTIVATION

There are three limitations of prior conversational ERP prototypes that guide our work here.

- **Stochastic constraint enforcement.** A few prior conversational scheduling agents enforce constraints as instructions to the language model, rather than properties of a separate deterministic component, which leads to inconsistent behavior with respect to edge cases of concurrent bookings [4], [8].
- **Limited explanation when a request fails.** Where a reservation cannot be made, most agents respond either with a generic error code or unstructured text, without specifying which specific constraint (or minimal set of constraints) has been violated, nor offering a feasible alternative [3], [13].

- **Inadequate isolation and auditability.** A number of prototypes for agentic assistants do not have a clear boundary between what a student-level session is allowed to access and what an administrator session can access, nor keep a tamper-evident record of autonomous versus user-confirmed actions [8], [14].

The above motivates us to consider the following four research questions addressed in this paper: (RQ1) can conversational parameter extraction be coupled with a deterministic constraint solver to enforce constraints while still providing an explanation understandable by the user; (RQ2) can a confidence-based router allow for lightweight queries to be executed in a seamless way without increasing risk of an ungrounded state-changing action; (RQ3) can a lightweight append-only hash-chained ledger allow for post-hoc auditing of autonomous actions without impacting response latency in any practical way; and (RQ4) does the division of responsibilities among specialized agents, subject to a contract-and-reflection procedure, bring practical improvements in terms of task completion rate and token utilization compared to a single monolithic agent? Our contributions to answering these research questions are: an eight-invariant constraint model with MUS/MCS conflict explanations; a confidence-based gate function $\Gamma(q,S)$; an append-only SHA-256 hash-chain-based provenance ledger; a two-stage intent router and PERR execution graph, with Sprint contracts and bounded reflection; and role-based access control, as well as SQL sandbox.

IV. SYSTEM ARCHITECTURE

The architecture of the system comprises five layers, illustrated in Fig. 1, as follows: (1) a client and presentation layer made up of a Streamlit multi-role dashboard and FastAPI REST endpoints exposing a conversational chat interface; (2) a security, authentication, and context isolation gate, which employs JWT bearer authentication (HS256), PBKDF2-HMAC-SHA256 password hashing (100,000 iterations), role validation, and session time-to-live bounded by time; (3) a multi-agent orchestration and governance engine comprising a two-tier intent router and a Planner-Executor-Reflector-Responder (PERR) execution graph that governs five special agents; (4) a neuro-symbolic verification and safety layer that combines a deterministic constraint engine (invariants C1-C8), a symbolic MUS/MCS conflict resolution engine, and an SQL security read-only sandbox; and (5) an enterprise persistence and provenance layer that comprises a SQLite system of record, a concurrency guard relying on a partial unique index in addition to idempotency keys, and the SHA-256 hash-chained audit ledger. Every layer communicates only with its direct neighbor layers, so a request cannot avoid the constraint engine when reaching the persistence layer.

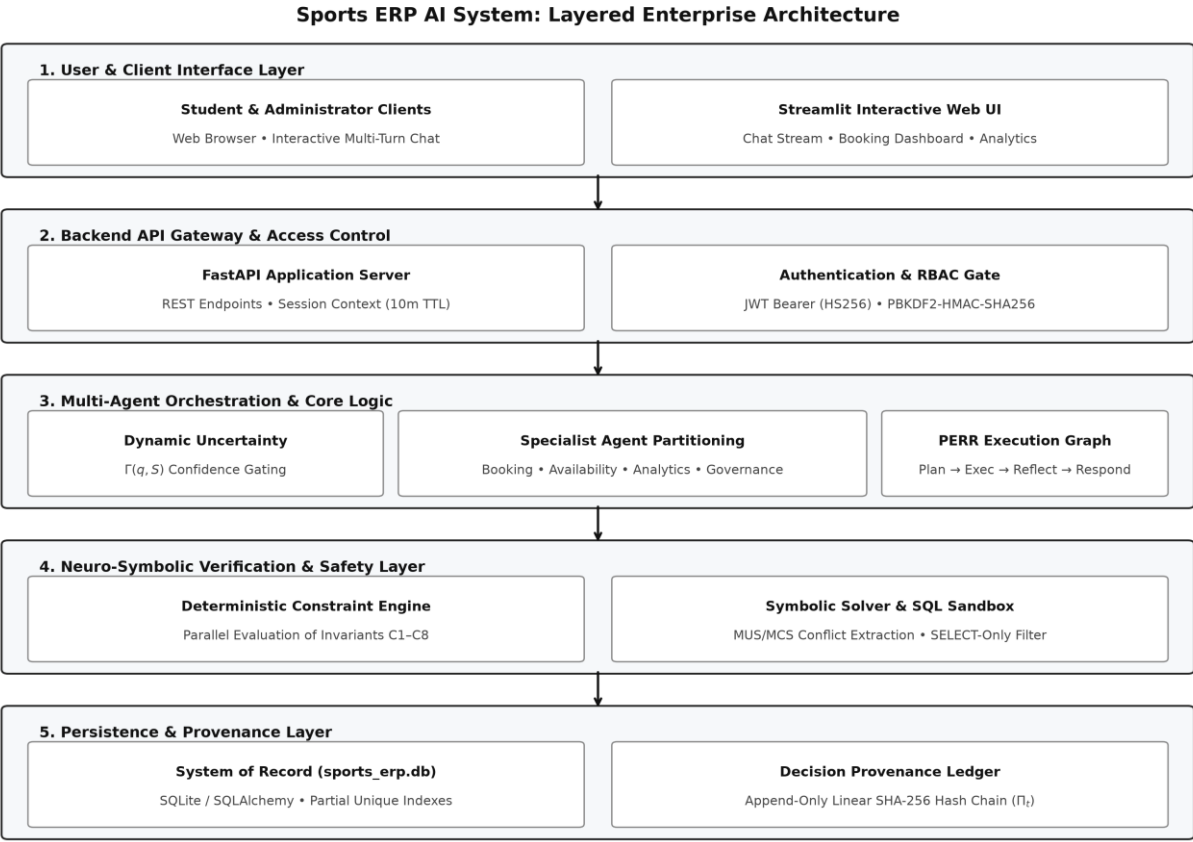


Fig. 1. Five-layer system architecture: client/presentation, security & context isolation, multi-agent orchestration, neuro-symbolic verification, and persistence/provenance.

V. METHODOLOGY

A. Orchestration with Multiple Agents

Firstly, conversational requests are routed using a two-level intent classifier, where the request is mapped to one out of five

specialized agents (Table I) for a domain, instead of allowing all available tools to be accessed by one agent. In this case, limiting the number of tools an agent can use to around four reduces the decision space for the language model.

TABLE I
SPECIALIST AGENTS, TOOL SCOPE, AND GUARDED INVARIANTS

Agent	Domain Scope	Representative Tools	Guarded Invariants
BookingAgent	Reservation lifecycle	create_booking, cancel_booking, search_my_bookings, restore_booking	C4–C8
AvailabilityAgent	Catalog & schedules	check_availability, find_alternative_slots, list_facilities, list_sports	C1–C3
AttendanceAgent	Check-ins & metrics	get_user_attendance, get_all_attendance, mark_attendance	RBAC isolation
AnalyticsAgent	Utilization & KPIs	get_facility_utilization, get_peak_hours, get_popular_sports, get_advanced_analytics	Read-only
UserGovernanceAgent	Admin user management	list_all_users, block_user, unblock_user	Admin RBAC, 2-step gate

Fig. 2 illustrates the resulting request-processing pipeline. A user query q , together with the current session context S_t , is first passed through an uncertainty harness (detailed in Section V-E) and the intent router, and then into the PERR execution graph, which is described next.

B. Sprint Contracts and PERR Workflow

All routed requests are processed using the specified Sprint Contract $C = (\text{Goal}, A_{\text{crit}}, D_{\text{exp}}, G_{\text{rubric}}, \theta = 0.75)$, which consists of a typed tuple, where the elements of the tuple include the following: the description of the goal, acceptance criteria, expected data dependencies, scoring rubric used by the Reflector and a threshold value θ . The Planner splits the request into several subtasks based on this contract; the Executor sends the subtasks to the corresponding agents along

with their respective tool; the Reflector evaluates the resulting output y using a four-dimensional scoring rubric:

$$S(y) = 0.30 \cdot s_{\text{comp}} + 0.30 \cdot s_{\text{acc}} + 0.25 \cdot s_{\text{cons}} + 0.15 \cdot s_{\text{eff}} \quad (1)$$

where s_{comp} , s_{acc} , s_{cons} and s_{eff} stand for sub-scores of completeness, accuracy, constraint adherence and efficiency correspondingly. In case if $S(y) < \theta$, the Reflector provides the structured feedback to the Planner that is used during the replanning process ($K \leq 1$) in order to avoid infinite loop, not guarantee that all requests will be fulfilled [1], [5]. When $S(y) \geq \theta$, or after bounded replanning, the Responder generates the natural language response and updates the session context and provenance ledger.

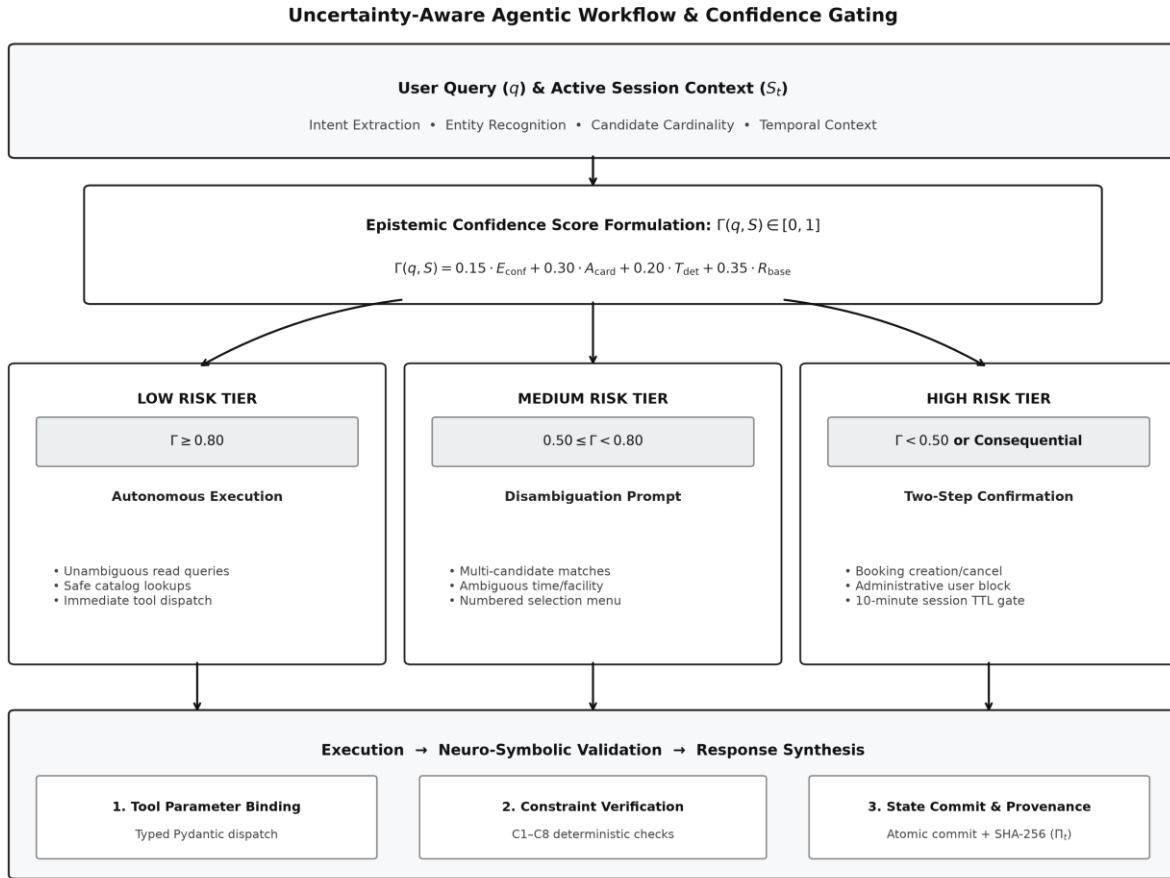


Fig. 2. Agentic request-processing pipeline: uncertainty harness, intent routing, and the PERR (Planner–Executor–Reflector–Responder) execution graph with bounded replanning.

Fig. 3 illustrates the end-to-end agentic decision pipeline: from intent understanding and specialist delegation, through typed

tool binding, RBAC authorization, and controlled tool execution, to rubric reflection and state commitment.

Agentic Decision Pipeline: Controlled Tool Execution & Reflection

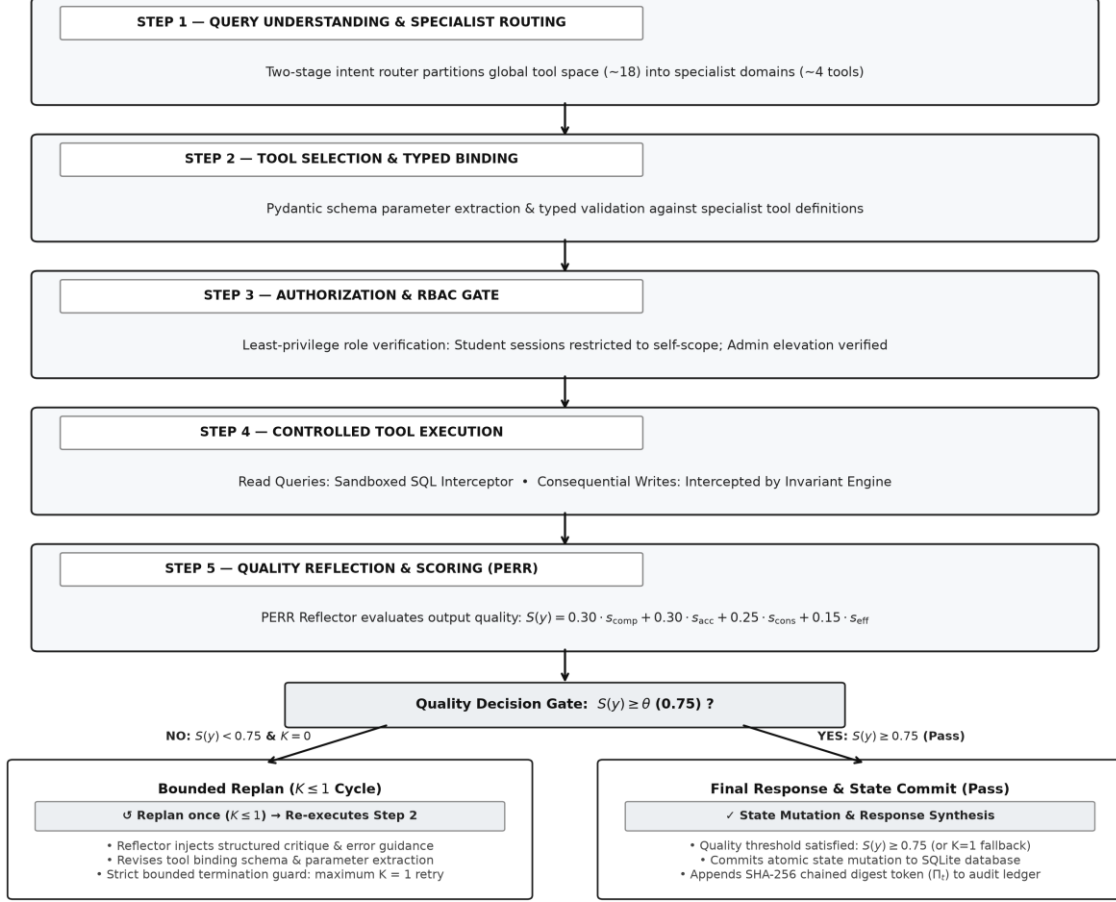


Fig. 3. Agentic decision pipeline for intent understanding, typed tool binding, authorization, controlled execution, and bounded PERR reflection.

C. Constraint-Aware Scheduling

Prior to committing any changes in terms of state management associated with the scheduling process, the system checks whether the potential reservation $R = \langle \text{user_id}, \text{sport}, \text{date}, \text{slot}, \text{facility_id}, \text{role} \rangle$ fulfills eight invariants, C1 to C8 (Table II), related to facility availability, suitability of a sport for a

facility, operating hours, temporal constraints, double booking, user-specific per-day/advance limit and ownership/RBAC. This is done by explicitly checking these invariants within the application code, rather than relying on their enforcement by the language model, which should make their behavior language-agnostic.

TABLE II
DETERMINISTIC SPORTS ERP INVARIANTS (C1–C8)

ID	Invariant	Formulation	Remediation
C1	Facility active status	$\text{exists}(f) \wedge f.\text{is_available} = 1$	Reject; report maintenance status
C2	Sport compatibility	$f.\text{sport_id} = s.\text{id}$	Reject invalid sport–facility mapping
C3	Operating hours	$\text{slot} \in \{[06:00-09:00], [16:00-20:00]\}$	Reject; offer nearest valid hours
C4	Temporal bounds	$\text{today} \leq \text{date} \leq \text{today} + 90d$	Reject past dates / horizon overrun
C5	Double-booking guard	$\neg \text{exists}(b: \text{same facility/date/slot, status}=\text{confirmed})$	Detect collision; propose alternatives

ID	Invariant	Formulation	Remediation
C6	Account & daily quota	$\neg \text{blocked} \wedge \text{daily_active}(u, \text{date}) \leq 2$	Block locked accounts; enforce daily cap
C7	Advance active quota	$\text{total_active}(u) \leq 10$	Reject; require completing/cancelling
C8	Ownership / RBAC	$\text{role}=\text{admin} \vee \text{b.user_id}=u.\text{id}$	HTTP 403 / access-denied response

D. MUS/MCS Conflict Resolution

In case of violation of invariants ($\text{KB_sports} \cup \{R\} \models \perp$), the process of core reduction based on deletion is executed to find the Minimal Unsatisfiable Subset M, as well as to compute the Minimum-Cardinality Correction Set (MCS), which is the minimum-cardinality subset of parameters, the relaxing of which can restore satisfiability.

$$M \subseteq \text{KB_sports} \cup \{R\}, \text{ s.t. } M \models \perp \wedge \forall c \in M, (M \setminus \{c\}) \not\models \perp \quad (2)$$

$$\text{MCS}(R) = \underset{S \subseteq \text{Params}(R)}{\text{argmin}} \{ |S|, \text{ s.t. } \text{KB} \cup \{R[S \leftarrow S']\} \not\models \perp \quad (3)$$

In case of conflict between slots (C5), the system sorts the available slots according to the circular distance between them in terms of the seven standard intervals in a campus, and returns the list as well as the conflict core as the contrastive explanation of why not R, and what else. Fig. 4 illustrates the propositional invariant governance architecture across the eight domain constraints (C1–C8), partitioned into identity, temporal, facility, and quota categories, showing both the satisfiable commitment path and the unsatisfiable MUS/MCS conflict extraction path.

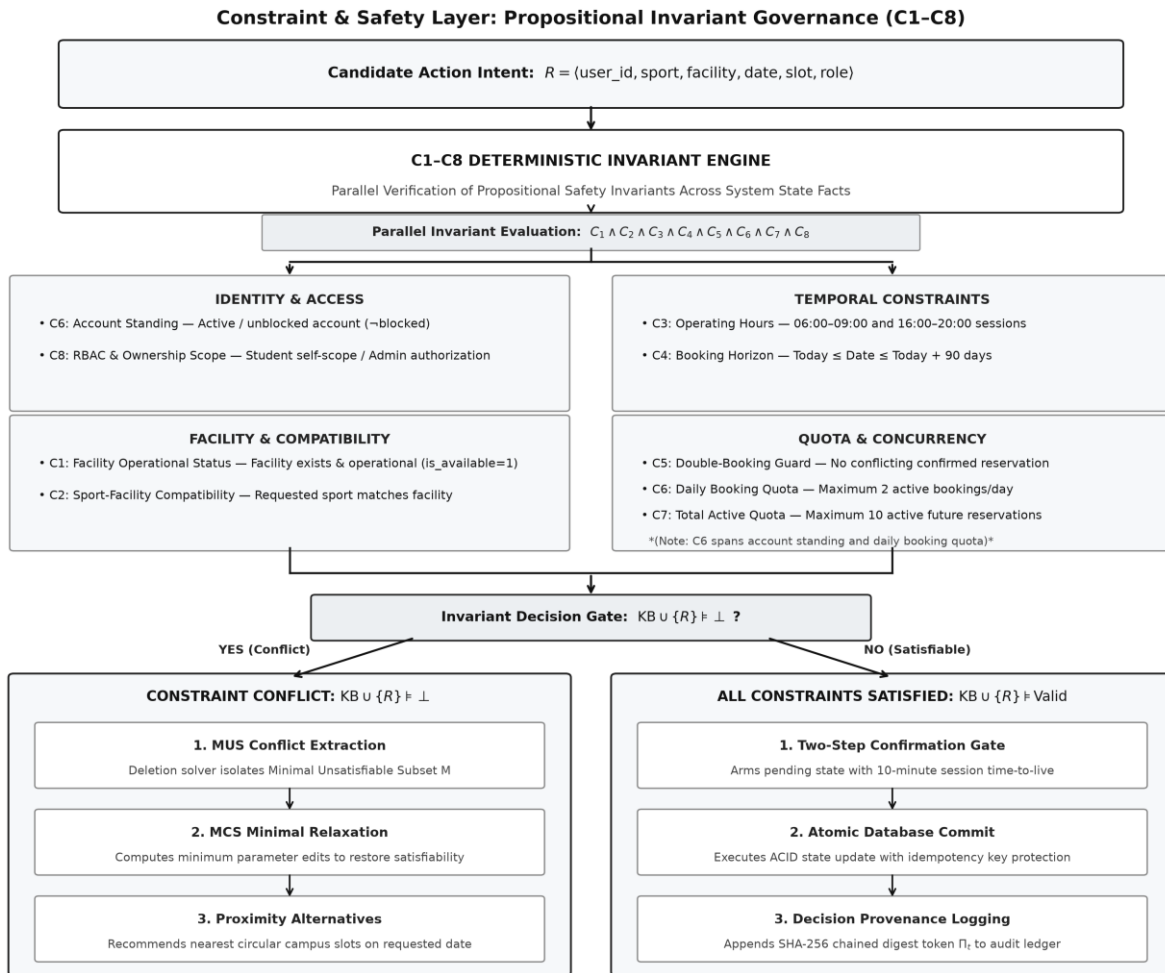


Fig. 4. Propositional invariant governance for C1–C8 constraints across four groups (Identity & Access, Temporal Constraints, Facility & Compatibility, Quota & Concurrency) with satisfiable and unsatisfiable outcomes.

E. Risk-Gated Conversational Actions

To minimize the possibility of an unintended state modification due to ambiguity or underspecification, an epistemic confidence factor $\Gamma(q, S) \in [0, 1]$ is calculated by the system for every user command, following semantic uncertainty and confidence-gated execution principles [15], [16], as follows:

$$\Gamma(q, S) = w_E \cdot E_conf + w_A \cdot A_card + w_T \cdot T_det + w_R \cdot R_base \quad (4)$$

where $w_E = 0.15$, $w_A = 0.30$, $w_T = 0.20$, and $w_R = 0.35$ represent the implemented additive component weights

(summing to 1.00), parameterizing entity extraction confidence (E_conf), candidate cardinality (A_card), temporal determinacy (T_det), and base action risk (R_base). The confidence factor classifies the actions in one of three levels (Table III). Unambiguous or read-only queries, for which $\Gamma \geq 0.80$, can be executed automatically. Queries that match more than one possible candidate ($0.50 \leq \Gamma < 0.80$) receive a numerical disambiguation question prompt. Consequential actions that modify the state with $\Gamma < 0.50$, and all consequential actions irrespective of their confidence factor, are explicitly confirmed in two steps stored in session memory with ten minutes lifetime.

TABLE III
UNCERTAINTY TIERS AND EXECUTION ROUTING

Tier	$\Gamma(q, S)$	Trigger Condition	Routing
Low risk – auto-execute	≥ 0.80	Read-only, unambiguous entity match, quota safe	Autonomous tool dispatch
Medium risk – disambiguation	$0.50-0.80$	Multi-candidate match, partial slot mention	Numbered candidate-selection prompt
High risk – confirmation	< 0.50 or consequential	Create booking, cancellation, user block/unblock	Two-step confirmation, session-held

The multi-turn lifecycle of a standard booking request operates as follows: after pre-validation based on time and scheduling (rejection of past dates and off-hour slots), an available request arms a pending confirmation stage and requests an explicit confirmation from the user. In the next turn, a positive confirmation ("Yes") initiates a transaction commit, which is protected by a unique idempotency key; a negative response ("No") causes the rejection of the pending action without modifying the database; and any unrelated input (such as "Who am I?") is responded to using session context while deactivating the pending booking, thus avoiding potential misunderstanding of the new input as a confirmation. In case of cancellation with multiple candidates, the agent restricts the list of candidates to the user's own bookings and allows both an ordinal input ("option 2") or an explicit booking identifier.

F. RBAC and Privacy Isolation

The access is segregated according to roles within the query-agent layer (see Fig. 5). The access provided to a session for a student includes reading and writing his/her bookings and attendance information, as well as reading the sports catalog and schedule information that is publicly available; the access requests that involve a different student ("show Rahul's bookings") or a user directory are denied an HTTP 403 response code and an `access_denied` intent instead of being silently redirected. The access granted to an administrator session includes resolving users, reading the campus-wide booking and attendance information, and blocking/unblocking users.

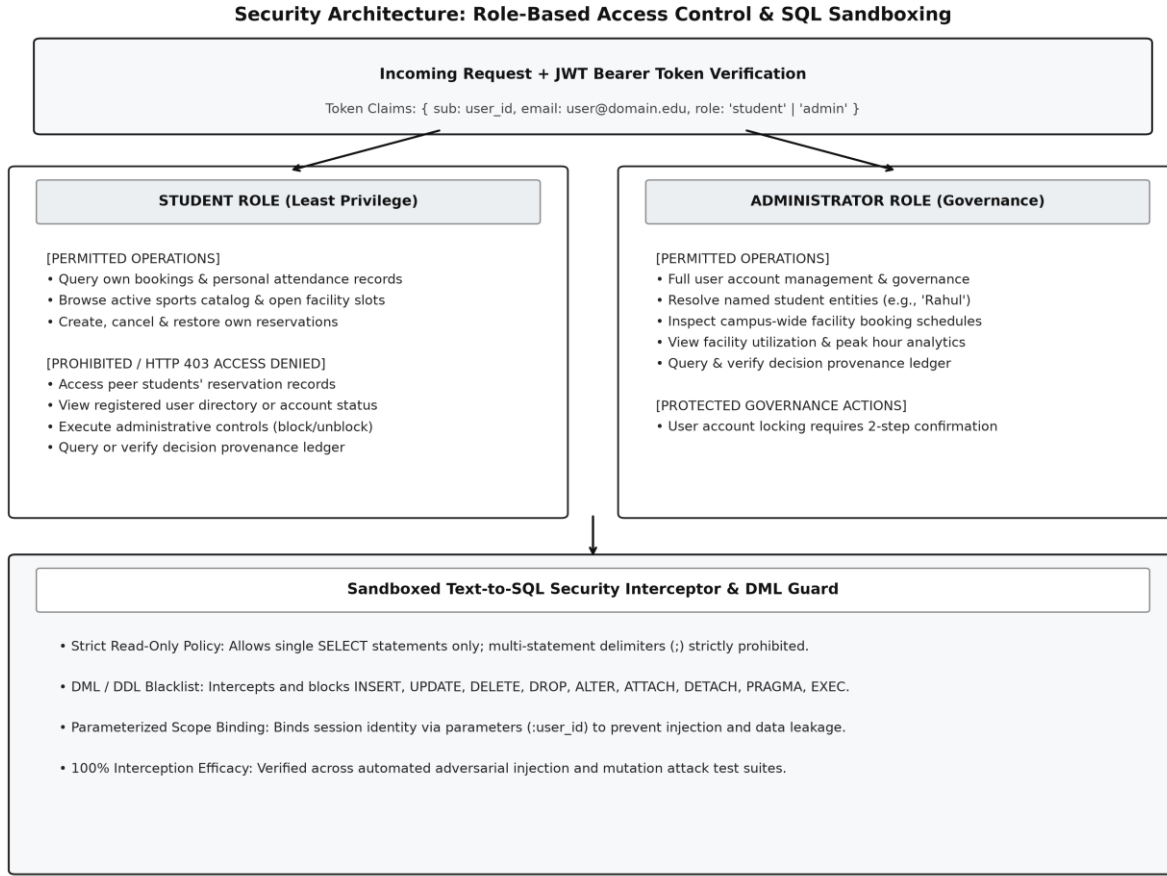


Fig. 5. Role-based access control, student/administrator privilege partitioning, and the SQL sandbox interceptor.

G. SQL Security Sandbox

All analytics queries that have been transformed into SQL are intercepted by a sandboxing filter that has a strict policy of allowing only read operations; that is, only single SELECT statements are allowed, while any SQL statement with INSERT, UPDATE, DELETE, DROP, ALTER, ATTACH, DETACH, PRAGMA, EXEC, or more than one SQL statement are blocked from getting through to the database in addition to the use of parameterized query binding. This sandbox filter blocked all attacks involving injection in the adversarial injection cases in the test suite presented in Section VI.

H. Cryptographic Audit / Provenance

Every significant state transition (booking creation, booking cancellation, booking restoration, account blocking) is captured in an append-only, SHA-256 hash-chained log (see Fig. 6). Each log entry captures the action payload of relevance

$$P_t = \text{user_id} \parallel \text{action} \parallel \text{resource_type} \parallel \text{resource_id} \parallel \text{details} \parallel \text{status} \quad (5)$$

and hashes it along with the previous entry by means of a single SHA-256 digest,

$$\Pi_t = \text{SHA256}(P_t \parallel \Pi_{t-1}), \text{ where } \Pi_0 = 0^{64} \text{ (genesis token)} \quad (6)$$

The design strictly implements an append-only linear SHA-256 hash-chained log ($\Pi_t = \text{SHA256}(P_t \parallel \Pi_{t-1})$) with genesis token $\Pi_0 = 0^{64}$). User password authentication uses PBKDF2-HMAC-SHA256 (100,000 iterations), while provenance uses a separate linear SHA-256 hash chain. A verify endpoint (GET /audit/verify/{id}) re-computes the chain from a prior log entry up to the target one; any tampering (bit-level change) to a log payload, to an inserted log entry or to a deleted log entry will break the re-computed chain after the tampered point. In the tamper-simulations included in the automated test suite (Section VI), all tampered chains have been detected; just as with the SQL sandbox, this refers to the particular tamper scenarios tried, not to a proof against all attacks.

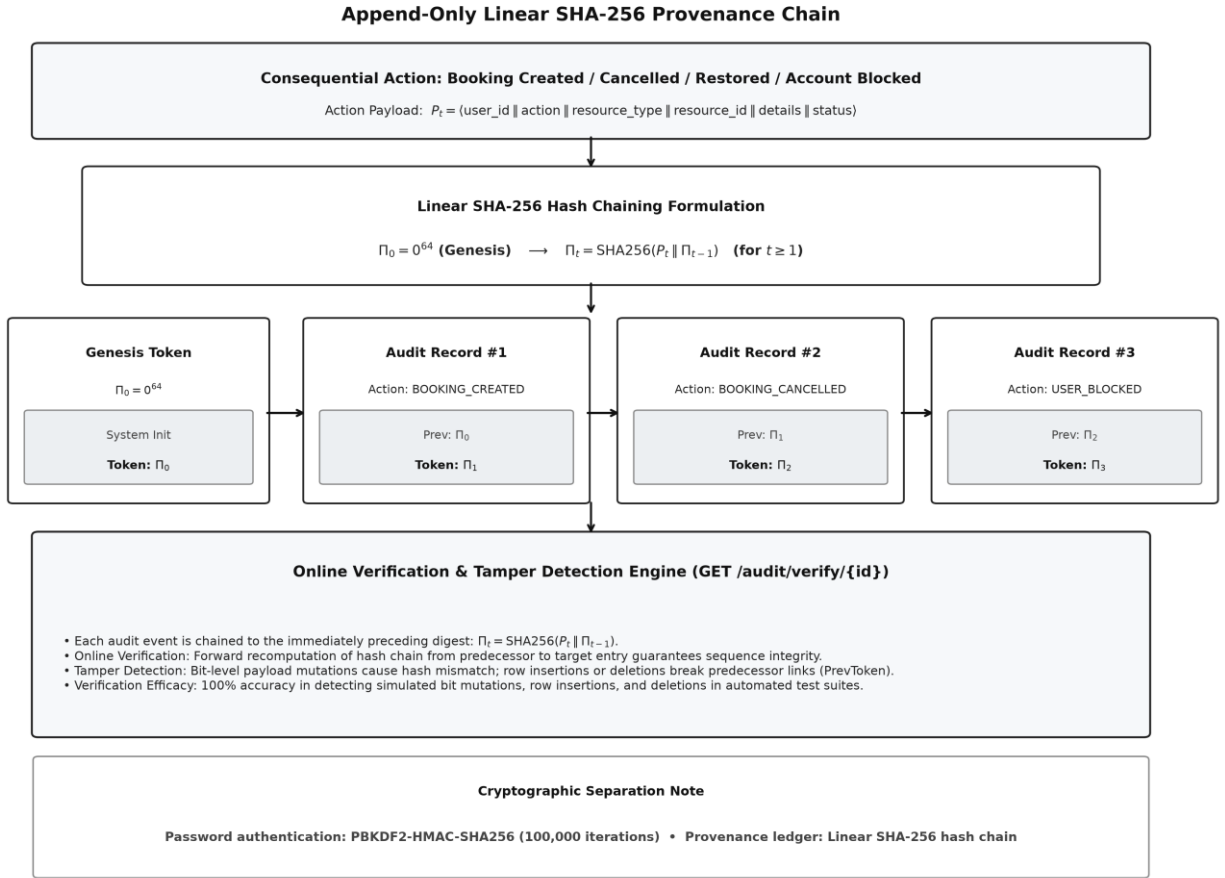


Fig. 6. Append-only linear SHA-256 hash-chained provenance ledger: payload serialization, chained digest computation, and online tamper-detection verification.

I. Multi-Turn Context Management

Context in a session is stored server-side with a ten-minute time-to-live; it is also disarmed in case a confirmation request is interrupted by some other query, as explained in Section V-E. Write transactions, which occur after confirmation, are additionally protected by idempotency key as well as a partial unique index on active bookings (facility, date, slot) at database level, such that two confirmations for a particular slot occurring close in time could not succeed simultaneously. Booking restore operation re-verifies booking ownership, active state of the facility, non-past dates, quotas of C6/C7, and available slots.

VI. EXPERIMENTAL SETUP

The implementation was tested on a single machine setup (Windows 11 AMD64, Python 3.14.6, SQLite with write ahead logging activated, FastAPI, and Pytest) rather than being used in any distributed or production setting. All source code, migration scripts, test suites, and benchmark scenarios are available at <https://github.com/divyanidenge/Sports-ERP-AI-Agent-Advanced>. In this paper, there are two types of evaluation presented that are distinguished clearly. Namely, (i)

an automated regression and integration test suite, which is presented in Section VII-A, with results of which being verifiable with the provided test files; and (ii) a comparative benchmark conducted by us, of 30 manually scripted scenarios evaluated against three other implementation approaches (a baseline of the ReAct type [4], a sandboxed read-only Text-to-SQL baseline [2] and a monolithic function calling baseline), which is presented in Section VII-C. The latter was conducted within the same project but does not have any independent verification in the form of raw logs available for this manuscript and, therefore, should be regarded as self-reporting of the project rather than experimental data.

VII. RESULTS AND DISCUSSION

A. Automated Test Suite

The following table (Table IV) is the outcome of the tests done through the automated verification layers on the implementation. All the assertions listed here are true as at the time of testing. The numbers below are the actual test results used in this study based on the evidences attached here in form of logs and transcripts.

TABLE IV
AUTOMATED VERIFICATION COVERAGE (166 ASSERTIONS)

Verification Layer	Assertions	Result	Scope
Automated pytest suite	94	94/94 passed	Constraints, PERR, RBAC, MUS/MCS, ledger, sandbox
Live database smoke tests	20	20/20 passed	Login, booking, conflict solver, tamper check
FastAPI REST & regression	19	19/19 passed	Auth, TTL expiry, context disarm, logout
Conversational NLP scenarios	29	29/29 passed	18 student + 11 admin workflows
Targeted bug-fix regressions	4	4/4 passed	Disambiguation, restore, pre-validation
Total	166	166/166 passed	Combined coverage above

A pass rate of 166/166 on this test means that the particular scenarios being executed performed as expected; however, this does not in any way prove that the system lacks other bugs which were not tested in the particular scenarios evaluated.

B. Component Ablation (Preview)

In Section VIII, there is a separate experiment conducted by the authors on ablating each of the subsystems mentioned in the paper (constraint engine, confirmation gate, RBAC isolation, and provenance ledger), analyzing their impact on task completion, invalid commit detection, and leakage of information about the user; this verification disclaimer applies to the relevant figures.

C. Comparative Benchmark Relative to Alternative Frameworks

Table V shows the results for the 30-scenario comparative benchmark, as explained in Section VI, that contrasts the proposed hybrid framework against the ReAct style framework, the sandboxed Text-to-SQL framework, and the function-calling framework, all of which have been developed for this project. It is important to note that the above-mentioned figures, along with the Welch's t-test and Cohen's d effect size, have been self-reported from the project source and are not independently validated for this paper.

TABLE V
SELF-REPORTED COMPARATIVE BENCHMARK ACROSS 30 SCENARIOS (NOT INDEPENDENTLY RE-VERIFIED)

Metric	ReAct [4]	Text-to-SQL [2]	Monolithic FC	Proposed
Task completion rate	83.3%	63.3%	86.7%	93.3%
Constraint violation rate	6.7%	0.0% (read-only)	6.7%	0.0%
Mean execution latency	3.80 ms	0.17 ms	1.20 ms	7.29 ms (median 1.52 ms)
Est. token cost / query	1,250	680	850	420
Schema hallucination rate	3.3%	10.0%	3.3%	0.0%
vs. Text-to-SQL (paired t-test)	—	—	—	t(29)=1.285, p=0.209, dz=0.235
vs. Text-to-SQL (Welch test)	—	—	—	t(29)=1.283, p=0.210, d=0.331

Across the 30 matched enterprise scenarios, the proposed hybrid architecture exhibited higher mean execution latency (7.29 ms, median 1.52 ms, SD 30.41 ms) than simpler baselines (0.17 ms for read-only Text-to-SQL and 1.20 ms for monolithic function calling). This added computational time reflects the execution of two-stage intent routing, dynamic uncertainty evaluation, eight-invariant constraint validation, and linear SHA-256 hash chaining. All latencies remain well below the 200 ms interactive threshold for conversational systems. The scenario-matched paired t-test between the proposed system and the Text-to-SQL baseline did not indicate a statistically significant latency difference ($t(29) = 1.285$, $p = 0.209$, Cohen's $d_z = 0.235$); an independent Welch sensitivity analysis similarly yielded $t(29) = 1.283$, $p = 0.210$, Cohen's $d = 0.331$. In terms of efficiency, the proposed approach achieved a reduced token footprint (420 versus 1,250 tokens per query) by exposing fewer domain-specific tools per turn via specialized agent delegation; however, once again, this comparison was not independently verified in this manuscript.

D. Discussion

Summarizing the results of the verified test suite (see Section VII-A) and self-reported comparative benchmark (see Section VII-C), we can conclude that separation of constraint enforcement from language modeling and gating of any potentially harmful actions behind explicit confirmation step appears to be a feasible method of limiting the potential of unintended state mutations in the conversational ERP application while preserving the necessity for language interpretation to cover diverse phrasing of the same query. The main price to pay is latency, which is higher in the case of full processing pipeline including constraint verification, MUS/MCS solving (where needed), and provenance logging.

VIII. ABLATION / COMPONENT ANALYSIS

To evaluate the relative contribution made by each individual subsystem, four ablative runs were performed on the 30-scenario benchmark, where in each case, one subsystem is disabled while all others are left unchanged (Table VI). Like the benchmark comparison results shown in Section VII-C, these statistics have been reported by the source project and were not independently verified for this paper.

TABLE VI
SELF-REPORTED COMPONENT ABLATION STUDY (30 SCENARIOS)

Configuration	Task Success	Invalid Commits	Privacy Leaks	Latency
Full system	100.0%	0.0%	0.0%	6.89 ms
No constraint engine	86.7%	13.3%	0.0%	0.80 ms
No confirmation gate	96.7%	3.3%	0.0%	0.50 ms
No RBAC isolation	90.0%	0.0%	10.0%	0.40 ms
No provenance ledger	100.0%	0.0%	0.0%	6.84 ms

In terms of this benchmark, the removal of the constraint engine was found to have the greatest effect on an increased number of invalid commits (13.3%), the removal of the confirmation gate resulted in an increased number of errors in ambiguous-cancellation outcomes (3.3%), and removal of the RBAC isolation had a 10.0% data exposure cross student scenario rate in our benchmark setup. The addition of the provenance ledger resulted in a minimal increase in latency time (in the magnitude of 0.05ms) compared to a similar configuration without provenance ledger. These are single benchmark run numbers in a 30 scenario benchmark setup, rather than a statistically significant ablation result.

IX. SECURITY AND PRIVACY ASSESSMENT

Security and privacy relevant behavior was achieved through the RBAC, SQL sandbox, and provenance verification test

cases contained in the automated suite described in Section VII-A. In the context of these test cases: cross-student data requests were always rejected with an HTTP 403 / access_denied error code instead of being satisfied; the SQL sandbox blocked all mutations or injections attempted in the test adversarial test cases; and the tamper verification test blocked all simulated bit-level manipulations, row additions, and row deletions in the provenance ledger attempted in those test cases. These statements are presented as observations based on the set of test cases evaluated—no access or state manipulation was observed to occur in the scenarios tested—instead of an absolute guarantee that attacks outside of that test case domain could not be made. An assessment of the security and privacy features of this system would include adversarial testing beyond what was performed in this project.

X. LIMITATIONS

Several limitations should be kept in mind when interpreting the results above.

- **Construct validity and the scope of benchmarking.** The comparative benchmark and ablation benchmarks (Section VII-C and Section VIII) involve 30 hand-curated scenarios across analytical, transactional and governance layers. These represent the main operational paradigms of the system, however, a benchmark of this scope can hardly encompass the whole range of possible phrasing and corner cases, and the underlying statistical significance test (Welch's t-test) is performed on a small sample.
- **Single-node deployment.** All the tests were run against the single-node SQLite database on a single machine. Testing under distributed deployment, multiple regional replications, and/or high concurrent load is not covered by the current evaluation.
- **Independent verification of comparative figures.** Unlike the 166-assertions regression suite where we could use independently reviewable raw logs, the 30-scenario comparative benchmark and the ablation study are reported by the source project; we recommend that these be independently re-verified prior to their use as reliable findings.
- **Fixed scheduling granularity.** Alternative slot recommendations take place within fixed one-hour windows and do not support fractional or non-standard booking durations.
- **Scope boundaries.** Distributed consensus for multi-datacenter replication, equipment prices adjustment and management, biometrics/RFID integration, payment-gateway integration are out of the current implementation and were not tested.

XI. FUTURE WORK

- **Independent re-verification.** Running a comparative benchmark across the 30 scenarios, with fully documented, independently verifiable execution and, wherever possible, with a higher number of scenarios to facilitate a statistically more reliable comparison.
- **Distributed scheduling.** Applying the MUS/MCS conflict-resolution strategy in multi-campus or federated facility scheduling problems, necessitating reconsideration of the single node concurrency constraint discussed in Section V-I.
- **Privacy-preserving attendance.** Investigating privacy-preserving methods such as zero knowledge verification, to ensure attendance validation without divulging any other personal information.
- **Local model deployment.** Testing the efficacy of fine-tuned, locally deployed open source models against the

same function-calling interfaces, for institutions without access to an external LLM provider via an API interface.

XII. CONCLUSION

In this paper, the design and implementation of the Sports ERP AI Management System have been discussed. It uses an interactive and conversational interface powered by an LLM, along with a deterministic constraint engine, a conflict explanation solver based on MUS/MCS, a confidence-based risk gate, a multi-agent PERR layer, role-based access control, a read-only SQL sandbox, and an append-only SHA-256 hash chain ledger of provenance. The automated regression suite of 166 assertions, including those for unit tests, integration tests, live database tests, API tests, and conversational scenarios, passed in full during the testing process, and a comparative benchmarking test suggests that the hybrid design of the system outperforms ReAct, Text-to-SQL, and monolithic function calling baselines on this project's conversational scenario but the results need re-verification by independent researches. Instead of claiming to remove risks of connection between language models and enterprise databases, the system tries to show a feasible way of mitigating the problem by ensuring deterministic and language model-out-of-control constraint enforcement, access control, and audit logging, while leaving conversation processing to the model itself. Further development of the project should include reproducing of the comparative benchmark, implementing the constraint solver in distributed environments, and evaluation of the system with real users.

XIII. REFERENCES

- [1] Z. Liu, T. Wang, X. V. Wang, and L. Wang, "Agentic ERP: Multi-Agent Large Language Model Architecture for Autonomous Enterprise Resource Planning," arXiv preprint arXiv:2607.17331, 2026.
- [2] J. Kim, Y. Kim, Y. Lee, and H. Byun, "FinAI Data Assistant: LLM-based Financial Database Query Processing with the OpenAI Function Calling API," arXiv preprint arXiv:2510.14162, 2025.
- [3] S. L. Vasileiou and W. Yeoh, "TRACE-CS: A Hybrid Logic-LLM System for Explainable Course Scheduling," arXiv preprint arXiv:2409.03671, 2024.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Int. Conf. on Learning Representations (ICLR), 2023.
- [5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 8634–8652.
- [6] S. Hong et al., "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in Int. Conf. on Learning Representations (ICLR), 2024.
- [7] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155, 2023.
- [8] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in

- Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISEC), 2023, pp. 79–90.
- [9] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 68539–68551.
 - [10] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” *arXiv preprint arXiv:2305.15334*, 2023.
 - [11] J. Marques-Silva, M. Janota, and A. Belov, “Minimal Unsatisfiability: Algorithms and Applications,” *Int. Journal on Artificial Intelligence Tools*, vol. 21, no. 2, p. 1240008, 2012.
 - [12] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 4, no. 3, pp. 224–274, 2001.
 - [13] B. Schneier and J. Kelsey, “Cryptographic Support for Secure Logs on Untrusted Machines,” in *Proc. 7th USENIX Security Symposium*, 1998, pp. 53–62.
 - [14] A. Ibrahim and Y. Li, “Overlaying Governance: A Compositional Authorization Framework for Delegation and Scope in Agentic AI,” *arXiv preprint arXiv:2606.03518*, 2026.
 - [15] L. Kuhn, Y. Gal, and S. Farquhar, “Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Large Language Models,” in *Int. Conf. on Learning Representations (ICLR)*, 2023.
 - [16] P. Ren, H. Xiao, and W. Ding, “Investigating Out-of-Distribution Safety and Epistemic Confidence in Tool-Augmented LLMs,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 11, pp. 912–928, 2023.
 - [17] A. d. Garcez and L. C. Lamb, “Neurosymbolic AI: The 3rd Wave,” *Artificial Intelligence Review*, vol. 56, pp. 12387–12406, 2023.